EX 6: BIDIRECTIONAL RNN VS FEEDFORWARD NN FOR TIME-SERIES PREDICTION

AIM

To implement a Bidirectional Recurrent Neural Network (BiRNN) and a Feedforward Neural Network (FFNN) for time-series prediction using the Airline Passenger dataset, compare their predictive performance using Mean Squared Error (MSE), and visualize the actual and predicted passenger counts.

PROCEDURE

1. **Import the required libraries, including Pandas, NumPy, PyTorch, Matplotlib, and scikit-learn.**
2. **Load the Airline Passenger dataset and normalize the passenger counts using MinMaxScaler.**
3. **Create input sequences of 10 consecutive time steps and use the next time step as the target value.**
4. **Split the sequences into training and testing datasets in an 80:20 ratio.**
5. **Convert the datasets into PyTorch tensors and prepare DataLoaders for training.**
6. **Define a Bidirectional RNN with 64 hidden units and a Feedforward Neural Network with 64 hidden neurons.**
7. **Train both models using the Adam optimizer and Mean Squared Error loss for 100 epochs.**
8. **Generate predictions using both trained models and transform the predictions back to the original passenger-count scale.**
9. **Plot the actual and predicted passenger counts and calculate the MSE for both models to compare their performance.**

CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error

# Load and preprocess dataset
url = (
    "https://raw.githubusercontent.com/"
    "jbrownlee/Datasets/refs/heads/master/"
    "monthly-airline-passengers.csv"
```

```
)

df = pd.read_csv(url, usecols=[1])
data = df.values.astype('float32')

scaler = MinMaxScaler(feature_range=(0, 1))
data_scaled = scaler.fit_transform(data)

# Create sequences
SEQ_LENGTH = 10

X = np.array([
    data_scaled[i:i + SEQ_LENGTH]
    for i in range(len(data_scaled) - SEQ_LENGTH)
])

y = np.array([
    data_scaled[i + SEQ_LENGTH]
    for i in range(len(data_scaled) - SEQ_LENGTH)
])

# Split dataset chronologically
train_size = int(len(X) * 0.8)

X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

# Convert to PyTorch tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)

X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)
```

```
train_loader = DataLoader(
    TensorDataset(X_train_tensor, y_train_tensor),
    batch_size=16,
    shuffle=True
)

# Define Bidirectional RNN
class BiRNN(nn.Module):
    def __init__(self, input_size=1, hidden_size=64):
        super(BiRNN, self).__init__()

        self.rnn = nn.RNN(
            input_size,
            hidden_size,
            batch_first=True,
            bidirectional=True
        )

        self.fc = nn.Linear(hidden_size * 2, 1)

    def forward(self, x):
        out, _ = self.rnn(x)
        return self.fc(out[:, -1, :])


# Define Feedforward Neural Network
class FeedforwardNN(nn.Module):
    def __init__(self, input_size=10):
        super(FeedforwardNN, self).__init__()

        self.fc1 = nn.Linear(input_size, 64)
        self.relu = nn.ReLU()
```

```
        self.fc2 = nn.Linear(64, 1)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        return self.fc2(x)


# Training function
def train_model(model, loader, epochs=100):
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=0.01
    )

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for seqs, targets in loader:
            optimizer.zero_grad()

            outputs = model(seqs)
            loss = criterion(outputs, targets)

            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        if (epoch + 1) % 20 == 0:
            print(
```

```
            f"Epoch {epoch + 1}/{epochs}, "
            f"Loss: {total_loss / len(loader):.5f}"
        )


# Evaluation function
def evaluate_model(model, X):
    model.eval()

    with torch.no_grad():
        return model(X).cpu().numpy()


# Train BiRNN
print("Training Bidirectional RNN")
birnn = BiRNN()
train_model(birnn, train_loader)

# Train FFNN
print("\nTraining Feedforward Neural Network")
ffnn = FeedforwardNN(input_size=SEQ_LENGTH)
train_model(ffnn, train_loader)

# Generate predictions
pred_birnn = evaluate_model(birnn, X_test_tensor)
pred_ffnn = evaluate_model(ffnn, X_test_tensor)

# Inverse transformation
pred_birnn_inv = scaler.inverse_transform(pred_birnn)
pred_ffnn_inv = scaler.inverse_transform(pred_ffnn)
y_test_inv = scaler.inverse_transform(y_test)

# Calculate MSE
```

```
mse_birnn = mean_squared_error(y_test_inv, pred_birnn_inv)
mse_ffnn = mean_squared_error(y_test_inv, pred_ffnn_inv)

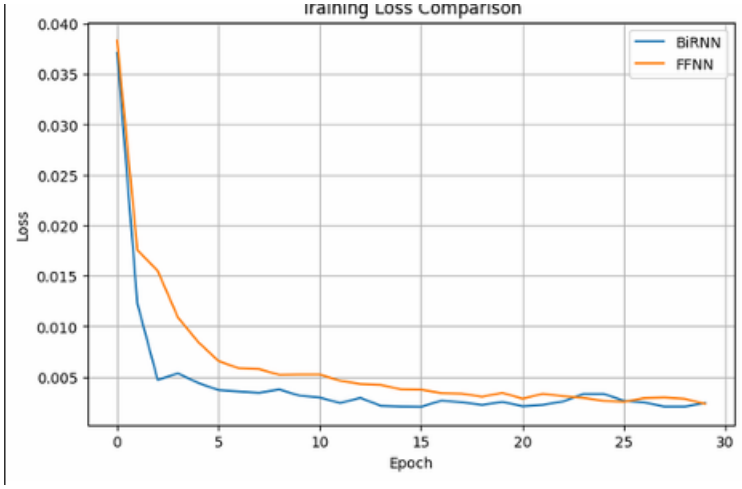
print(f"\nBiRNN MSE: {mse_birnn:.3f}")
print(f"FFNN MSE: {mse_ffnn:.3f}")

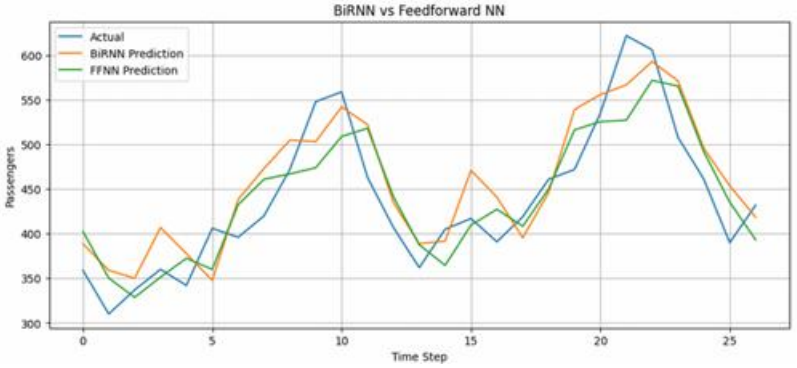
# Plot actual and predicted values
plt.figure(figsize=(12, 5))

plt.plot(y_test_inv, label='Actual')
plt.plot(pred_birnn_inv, label='BiRNN Prediction')
plt.plot(pred_ffnn_inv, label='FFNN Prediction')

plt.title('BiRNN vs Feedforward NN on Airline Passenger Data')
plt.xlabel('Time Step')
plt.ylabel('Passengers')
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT





RESULT

Thus, a Bidirectional Recurrent Neural Network (BiRNN) and a Feedforward Neural Network (FFNN) were successfully implemented using PyTorch for time-series prediction on the Airline Passenger dataset. Their predictions were visualized and evaluated using Mean Squared Error (MSE). The experiment demonstrated the differences between recurrent and feedforward architectures in learning temporal patterns and forecasting passenger counts.